

INDEX

Roll No.: VTU30/61

Sub

Roll No. VTC50181			Page No. Marks	Teacher's Sign / Remarks
S.No.	Date	Title		
1	23/07/25	Introduction to Python commands	15	
2	30/07/25	Implement conditional and looping statements	15	
3	06/08/25	importing and creating Python modules and packages in Python program	15	
4	13/08/25	use various datatypes, list, tuples and dictionary in Python programming	15	
5	20/08/25	implement various searching and sorting operations in Python	15	
6	03/09/25	utilizing 'functions' concepts in Python programming	15	
7	10/09/25	implement Python generator and decorators	15	
8	18/09/25	implement various txt/csv file operations	15	
9	24/09/25	implement Exceptions and Exceptional handling	15	
10	8/10/25	use matplotlib module for plotting in Python	15	
11	15/10/25	use tkinter module for UI design	15	
12	15/10/25	simulate gaming concepts using Pygame	15	
<i>Completed</i>				

23/07/25

Task-1 Running python script and various expression in an interactive interpreter key terms covered introduction to python commands, scripts

Tags - easy C0, S3 1.1 karan spent ₹150 on books ₹220 on groceries and ₹90 transport. Help him calculate the total expenses.

Aim:- To write a python program that calculates the total amount spent by karan on books groceries and transport

Algorithm:-

1. start the program
2. Accept the amount spent on books, groceries and transport
3. calculate the total expenses by summing all three amounts
4. Display the total amounts spent
5. End the program

Result:- Thus the amount spent by karan on books groceries and transport are proved

✓

Python Program

Program to calculate total expenses of karan

step 1 : Assign expenses

books = 150

groceries = 220

transport = 90

step 2 : calculate total

total expenses = books + groceries + transport

step 3 : Display the result

print ("Total expenses incurred by karan : ₹ :
total - expenses)

Input:-

Books = ₹150

Groceries = ₹220

Transport = ₹90

Output:-

Total expenses incurred by karan = ₹ 460

1-2

Aim:- To write a Python Program that calculates and displays the Body Mass Index (BMI) of a person using their weight (In kgs) & height (in meter)

Algorithm:-

1. start the Program
2. Prompt the user to input their weight in kg
3. Prompt the user to input their height in meters
4. calculate BMI using the formula
$$BMI = \frac{\text{weight}}{\text{height}^2}$$
5. Display the calculated BMI
6. End the Program.

Result:- Thus the body mass index of a person using their weight (kg) and height (m) are proved



Python Program:-

BMI Calculator

Step 1: Get input from the user

weight = float(input("Enter your weight in kg"))

height = float(input("Enter your height in meters"))

Step 2: Calculate BMI

$BMI = \text{weight} / (\text{height}^2)$

Step 3: Display result

print("Your Body mass index (BMI) is: ",
round(bmi, 2))

Input:

Enter your weight in kilogram : 70

Enter your height in meters : 1.75

Output:

Your Body mass Index (BMI) is : 22.86

1.3

Aim:- To write a Python program to find the area of the triangle when the length of all three sides are given using Heron's formula.

Algorithm:-

1. start the program
2. Accept the length of the three sides a, b and c
3. calculate the semi perimeter
$$s = \frac{a+b+c}{2}$$
4. using Heron's formula calculates
$$\text{area} = \sqrt{s(s-a)(s-b)(s-c)}$$
5. Display the area of the triangle
6. End the Program

VEL TECH	
EX NO.	1
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	
TOTAL (20)	
SIGN WITH DATE	15

Result:- Thus the area of triangle when the length of all three sides are proved by Heron's formula.

Python Program:-

Import math

step-1: Assign side lengths

$$a=8 \quad b=6 \quad c=4$$

step-2: calculate semi perimeter

$$s = (a+b+c)/2$$

step-3: Apply Heron's formula

$$\text{area} = \text{math.sqrt}(s * (s-a) * (s-b) * (s-c))$$

step-4: Display result.

print ("the area of the triangle is "

round (area, 2) "square cm")

Sample Input

values of the triangle are assigned

side $a=8\text{cm}$

$b=6\text{cm}$

$c=4\text{cm}$

Sample output

The area of triangle is : 11.62 square cm

30/07/25

Task 2 :- Implement conditional and looping statements.

Aim:- To implement conditional, control and looping statements using Python

Algorithm:-

1. start
 2. Get the input mark from the user
 3. with the use of an if-elif-else statement do
 - If the marks ≥ 90 Print grade "A"
 - If the marks is between 80 and 89 Print grade "B"
 - If the mark is between 70 and 79 Print grade "C"
 - if the mark is between 60 and 69 Print grade "D"
 - If the mark is below 60, Print grade "F"
 - stop
- 

Program:-

```
score = input(input("Enter the score:"))
```

```
if score >= 90
```

```
    Print ("The Grade is A")
```

```
elif (score <= 89 and score >= 80):
```

```
    Print ("The Grade is B")
```

```
elif (score <= 79 and score >= 70):
```

```
    Print ("The grade is C")
```

```
elif (score <= 69 and score >= 60):
```

```
    Print ("The Grade is D")
```

```
else:
```

```
    Print ("The Grade is F")
```

Output:- Enter the score :60
The Grade is D

2.2

Aim: To write a Python program that uses ladderized if elif-else statements

Algorithm:

- Accept battery percentage from the user
- use ladderized if elif else to determine health category
- if Percentage $\geq 90 \rightarrow$ Excellent Battery Health
- if $70 \leq \text{Percentage} < 90 \rightarrow$ Good Battery Health
- if $40 \leq \text{Percentage} < 70 \rightarrow$ Average Battery Health
- if Percentage $< 40 \rightarrow$ Poor Battery Health

Program:-

```
# Battery Health checker
Percentage = int(input("Enter battery percentage:"))
if Percentage >= 90:
    print("Excellent Battery Health")
elif Percentage >= 70:
    print("Good Battery Health")
elif Percentage >= 40:
    print("Average Battery Health")
else:
    print("Poor Battery Health")
```

input:-

Battery charge Percentage (integer)

output:-

Enter battery Percentage: 85

Good battery Health



2.3

Aim:- To determine wheather each of 5 visitors is allowed to ride at an amusement park based on a minimum height requirement of 120cm

Algorithm:-

- start the program
- set the total number of visitors to 5
- Loop from visitor 1 to visitor 5
 - Accept the height of visitor as input
 - if height is greather than or equal to 120 cm
 - Print Allowed Else print Not Allowed
- End the loop after 5 visitors have been checked
- stop the program.

Program:-

```
for i in range (1,6):
```

```
height = int (input (f " Enter height of visitor {i} in (cm.)"))
```

```
if height >= 120
```

```
    Print ("Allowed to ride.")
```

```
else Print ("Not Allowed to ride.")
```

VEL TECH	
EX NO.	2
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	
TOTAL (20)	
SIGN WITH DATE	15

Result:- Thus, The Python Program was successfully implemented using conditional statements (if-else) control flow and looping statements.

Input:

Enter height of visitor 1 in cm: 130

Enter height of visitor 2 in cm: 110

Enter height of visitor 3 in cm: 150

Enter height of visitor 4 in cm: 90

Enter height of visitor 5 in cm: 125

Sample output:

Allowed

Not allowed

Allowed

Not allowed

Allowed



06/08/25
Task-3: Importing and creating python modules and packages in python program.

Aim:- To implement and demonstrate the process of implementing built in modules, creating user-defined modules and organizing code into packages in python, thereby promoting code reusability, modularity and maintainability

Algorithm:

- import required modules; math, random, os, sys,
 - Math & random ;
 - compute $\sqrt{x}$, radians (30) a random float in $[0, 0.1, 0]$ a random integer in $[2, 6]$ (inclusive) π , $\text{ceil}(2, 3)$ $\text{floor}(2, 3)$ factorial (5), gcd (5, 15), abs (-10), Pow (3, 5), log base 3 of 2, $\log_{10}(a)$ for $a=100$, and check NaN/infinity.
 - os & ~~sys~~ sys
 - create c:\pythonlab if not Present and print the current working directory
 - create c:\pythonslots2L4 if not Present and change the current working directory to it
 - List all files / directories in new current directory
 - Print Python interpreter version.
 - statistics
 - on lists [5, 6, 8, 10] and [2, 5, 3, 2, 8, 3, 9, 4, 2, 5, 6],
 - compute mean, median, mode, stdev
 - Print neatly formatted results.
-

Program:-

```
import math
import random
import os
import sys
import statistics as stats
from pathlib import Path

Print("\n -- MATH & RANDOM --")
Print("sqrt(5) =", math.sqrt(5))
Print("radians(30) =", math.radians(30))
Print("randint(2,6) =", random.randint(2, 6)) # inclusive
Print("pi =", math.pi)

Print("ceil(2.3) =", math.ceil(2.3))
Print("floor(2.3) =", math.floor(2.3))
Print("factorial(5) =", math.factorial(5))
Print("gcd(5, 15) =", math.gcd(5, 15))
Print("abs(-10) =", abs(-10))
Print("abs(-10) =", abs(-10))

a_val = 100
Printf("log10 {a_val} =", math.log10(a_val))
inf_val = float('inf')
nan_val = float('nan')
Printf("f'is inf (∞) = {math.isinf(inf_val), isnan(nan_val)}
      = {math.isnan(nan_val)}")

Printf("\n -- OS & SYS --")
Path = Pythonlab = Path(r"C:\Pythonlab")
Path = Pythonlab.mkdir(parents=True, exist_ok=True)
Printf("f'created/ensured: {(Path - Pythonlab)}")

Print("Current working directory:", os.getcwd())
```

```
target_dir = path(r"C:\Python\slot s2L4")
target_dir.mkdir(parents=True, exist_ok=True)
os.chdir(target_dir)
print(f"changed into : {target_dir}")
print("Directory contents:", os.listdir())
print("Python version:", sys.version)
print("\n--- STATISTICS ---")
data1 = [5, 6, 8, 10]
data2 = [2, 5, 3, 2, 8, 3, 9, 4, 2, 5, 6]
print(f"mean ({data1}) = ", stats.mean(data1))
print(f"median ({data1}) = ", stats.median(data1))
print(f"mode ({data2}) = ", stats.mode(data2))
print(f"stdev ({data2}) = ", stats.stdev(data2))
```



output:

---MATH & RANDOM---

$$\text{sqrt}(5) = 2.23606797749979$$

$$\text{radians}(30) = 0.5235987755982988$$

$$\text{random}() \text{ in } [0,1) = 0.37444887175646646 \quad \leftarrow \text{will vary}$$

$$\text{randint}(2,6) = 6$$

$$\pi = 3.141592653589783$$

$$\text{ceil}(2.3) = 3$$

$$\text{floor}(2.3) = 2$$

$$\text{factorial}(5) = 120$$

$$\text{gcd}(5,15) = 5$$

$$\text{abs}(\overline{5+15}) = 10$$

$$\text{pow}(3,5) = 243$$

$$\log_{\text{base } 3} \text{ of } 2 = 0.6309297535714574$$

$$\log_{10}(100) = 2.0$$

$$\text{isinf}(\infty) = \text{True}, \text{isnan}(\text{NaN}) = \text{True}$$

---os & sys---

created / ensured; | Python lab

current working directory c: | --- (your current path)

created / ensured & changed into c: | Python\slot s2L4

directory contents of c: | Python\slot s2L4; [

Python version: 3.x.x --- (- details ---)

---STATISTICS---

$$\text{mean}([5,6,8,10]) = 7.25$$

$$\text{median}([5,6,8,10]) = 7.0$$

$$\text{mode}([2,5,3,2,8,3,9,4,2,5,6]) = 2$$

$$\text{stdev}([2,5,3,2,8,3,9,4,2,5,6]) = 2.271563383201093$$

3.2 Aim: To create a program to simulate a deck of cards shuffling and display the cards randomly

Algorithm:

step 1: start

step 2: To create a package cardpack

step 3: To create a module cardfun and import random function

step 4: Assign a cards range

step 5: call a module function

step 6: Display the random sample cards

step 7: stop

Program:-

```
cardfun
import random
def func():
    cards = []
    for i in range(1,53):
        cards.append(i)
    shuffled_cards = random.sample(cards, k=52)
    printf("\n\n", shuffled_cards, "\n\n")
Mymod.py
import cardfun
cardfun.func()
```

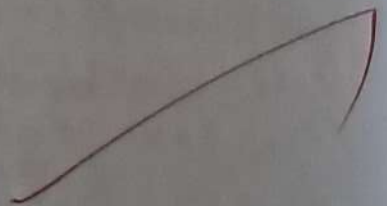
Output:-

RESTART

C:\users\student.MAT2V.C6833\APP Data\Local\Programs\Python\Python311\Lib\site-packages\CardPack\My Mod.Py

[5, 24, 13, 22, 20, 41, 38, 51, 4, 7, 34, 49, 14, 50, 37, 40,
15, 35, 17, 18, 33, 39, 36, 42, 12, 6, 16, 19, 48, 29, 2, 27, 11,
31, 46, 28, 21, 32, 8, 25, 30, 23, 26, 10, 43, 47, 3, 44, 52
1, 45, 9]

((state)) state = (state) name
((state)) name, state = (state) name
((state)) state = (state) name
((state)) state = (state) name



3.3

Aim:- To create a program to perform arithmetic operations using a custom module, handling division by zero errors

Algorithm:-

- step 1:- Define functions for $+$, $-$, $*$, $\div$.
- step 2:- Handle division by zero by raising an error if divisor is zero
- step 3:- import the module containing these functions
- step 4:- initialize two numbers ($a=10$; $b=5$)
- step 5:- call each function using `my math < function name >(a,b)`
- step 6:- print the result all operations

Program:- (my) math

```
def add(a,b):
```

```
    return a+b
```

```
def subtract(a,b):
```

```
    return a-b
```

```
def multiply(a,b):
```

```
    return a*b
```

```
def divide(a,b):
```

```
    if b==0:
```

```
        raise ValueError("cannot divide by zero")
```

```
    return a/b
```

```
import mymath
```

```
a=10
```

```
b=5
```


output:

Addition : 15

Subtraction : 5

Multiplication : 50

Division : 2.0

13/08/25 Task-4 - use various data types ; list, Tuples and Dictionary in Python Programming.

Aim:- To use various datatypes, tuples, and Dictionary in Python programming

Algorithm:-

1. Add elements to the list
2. Remove specific elements from list
3. sort list in ascending and descending order
4. find minimum and maximum elements in the list
5. calculate the sum and average of elements in the list
6. Print output
7. End

Program:-

```
# Add elements
```

```
list [10, 20]
```

```
a = 30
```

```
list.append(a)
```

```
Print(list)
```

```
# Remove elements
```

```
list.pop(1) # by index value
```

```
Print(list)
```

```
list.remove(10) # by item name
```

```
Print(list)
```

```
# sort elements : sort in ascending and descending
```

```
l = [5, 8, 9, 15, 30, 89]
```

```
Print(sorted(l))
```

output:-

[10, 20, 30]

[10, 30]

[30]

[5, 8, 9, 15, 30, 89]

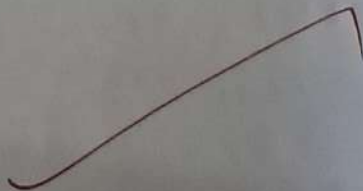
The minimum is : 5

The maximum is : 89

The sum is : 156

The average is : 26.0

VELT	NO
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	10



```
Print ("The minimum value is: "min(l))
```

```
Print ("The maximum value is: "max(l))
```

```
Print ("The sum is: ", sum(l))
```

```
Print ("The average is: ", (sum(l)/len(l)))
```

4.2

Aim: To create a Program to demonstrate tuple operations, including creation, elements access, concatenation and handling immutability

Algorithm:

1. start
2. To create a tuple use "tuple-name = (values)";
3. To access the elements of a tuple either use the index values
4. To concatenate tuples use operator "+" (tuple1 + tuple2)
5. Try to modify the tuple elements by assigning the values directly like: tuple (index) = new-value will result in an error as it is immutable
6. Print the output
7. End

Program:

```
# create a tuple : Define a tuple with elements of different data types (10, 'hello', 3.14, 'world')
```

```
tuple = (10, 'hello', 3.14, 'world')
```

```
Print (tuple)
```

```
for i in tuple
```

```
Print (i)
```

```
Print (tuple [1:3])
```

```
Print (tuple [1])
```

```
t2 = (5, 0.5)
```

```
t3 = (tuple + t2)
```

```
Print (t3)
```

```
tuple (3) = "PI" # ERROR
```

4.3

Aim:- To Create a Python Program to demonstrate dictionary operations, including creation, value access, modification and iteration

Algorithm:-

1. start the Program
2. Define a dictionary with key value pairs of different data types
3. Retrieve value from dictionary using their corresponding keys
4. Modify Dictionary
5. iterate over Dictionary
6. stop the Program.

Program:-

```
#create a Dictionary
```

```
types = {'name': 'Alice', 'age': 30, 'city': 'New York'}
```

```
dictionary = {'name': 'Alice', 'age': 30, 'city': 'New York'}
```

```
Print(dictionary)
```

Output:-

(10, 'hello', 3.14, 'world')

10

hello

3.14

world

('hello', 3.14)

(10, 'hello', 3.14)

output:-

```
{'name': 'Alice', 'age': 30, 'city': 'New York'}
```

Alice

30

```
{'name': 'James', 'age': 30, 'city': 'New York'}
```

```
{'name': 'James', 'age': 30}
```

KEY: name

KEY: age

dict-items([('name', 'James'), ('age', 30)])

Print (dictionary) ['name']

Print (dictionary) [age]

existing pairs

dictionary ['name'] = "James"

Print (dictionary)

dictionary.pop ('city')

Print (dictionary)

for k in dictionary:

Print ("key:", k)

Print (dictionary.item ())

VEL TECH	
EX NO.	9
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	
TOTAL (20)	15
SIGN WITH DATE	

Result: Thus, various data types, list, Tuples
and Dictionary in Python Programming was used
and verified successfully

90/08/25 Task-5- Implement various searching and sorting operations in Python programming.

Aim:- To implement various searching and sorting operations in Python Programming

Algorithm:

1. Input definition
2. Define the function find-employee-by-id that takes two Parameters:
 - a. A list of dictionaries (employees); where each dictionary represent an employee record with keys id, name and department
 - b. An integer representing the employee id to be searched
3. iterate through the list:
use for loop to iterate through each dictionary in employee list
4. check for matching ID:
within the loop; check if the id field of current dictionary matches the target id
5. Return id Matching:
if a match is found, return the current dictionary
6. Handle no match:
if loop completes without finding a match, return None.

Program:

```
def find-employee-by-id(employee, target-id):  
    for employee in employees:  
        if employee['id'] == target-id:  
            return employee  
#Test the function  
employees = [  
    {'id': 1, 'name': 'Alice', 'department': 'HR'},  
    {'id': 2, 'name': 'Bob', 'department': 'Engineering'},  
    {'id': 3, 'name': 'Charlie', 'department': 'Sales'}  
]  
Print.(find-employee-by-id(employees, 2)) #output:  
{'id': 2, 'name': 'Bob', 'department': 'Engineering'}
```

5.2

Aim: To implement a bubble sort algorithm to sort a list of students based on their scores in ascending order

Algorithms:-

1. Initialize n as the length of students list.
2. Iterate through the list $n-1$ times
3. in each iteration compare adjacent elements and swap if necessary
4. Track swaps and break the loop if no swaps are made
5. The list is sorted in ascending order by score.

Output:

```
{ 'id': 2, 'name': 'Bob', 'department': 'Engineering' }
```

Program:

```
def bubble_sort_scores(students):  
    n = len(students)  
    for i in range(n):  
        # track if any swap is made in this pass  
        swapped = False  
        for j in range(0, n-i-1):  
            if students[j]['score'] > students[j+1]['score']:  
                # swap if the score of the current student is  
                # greater than the next  
                students[j], students[j+1] = students[j+1], students[j]  
                swapped = True  
        # if no two elements were swapped, the list is already  
        # sorted  
        if not not sorted swapped:  
            break  
  
# Example usage  
students = [  
    {'name': 'Alice', 'score': 88},  
    {'name': 'Bob', 'score': 95},  
    {'name': 'Charlie', 'score': 75},  
    {'name': 'Diana', 'score': 85},  
]  
Print("Before sorting:")  
    for student in students:  
        Print(student)
```

output:

Before sorting:

{ 'name': 'Alice', 'score': 88 }

{ 'name': 'Bob', 'score': 95 }

{ 'name': 'Charlie', 'score': 75 }

{ 'name': 'Diana', 'score': 85 }

After sorting:

{ 'name': 'Charlie', 'score': 75 }

{ 'name': 'Diana', 'score': 85 }

{ 'name': 'Alice', 'score': 88 }

{ 'name': 'Bob', 'score': 95 }

bubble = sort - scores (students)

Print ("In After sorting: ")

for student in students :

Print (student)

VEL TECH	
EX NO.	
PERFORMANCE (5)	✓
RESULT AND ANALYSIS (5)	✓
VIVA VOCE (5)	✓
RECORD (5)	✓
TOTAL (20)	
DATE	15

Results Thus the program for various searching and sorting operations is executed and verified successfully

03/09/25 Task 6: Utilizing 'Functions' concepts. in Python Programming

Aim: To write Python programming using 'Functions' concept in Python Programming

Algorithm:-

1. Print welcome message
2. Determine number of students using `len()`
3. Print types of student lists using `type()`
4. Find highest and lowest grades using `max()` and `min()`
5. Print sorted and reversed grade lists using `sorted()` and `reversed()`
6. Generate grade indices using `range()`
7. stop

Program:-

```
def analyze_student_grades():  
    student_names = ["Alice", "Bob", "Charlie", "Diana"]  
    student_grades = [85, 92, 78, 90]  
  
    Print ("Welcome to the student grades Analyzer!\n")  
    num_students = len (student_names)  
    Print ("Number of students:", num_students)  
    Print ("In Type of student_names_list:", type(student_names))  
    Print ("Type of student_grades_list:", type(student_grades))  
    highest_grade = max(student_grades)
```

output:

welcome to the student Analyzer!

Number of students : 4

Type of student_names list : <class 'list'>

Type of student_grades list : <class 'list'>

Highest grade : 92

lowest grade : 78

Sorted grades : [78, 85, 90, 92]

reversed grades : [92, 90, 85, 78]

Grade indices from 1 to number of students : [1, 2, 3, 4]

lowest_grade = min(student_grades)

Print ("In Highest grade:", highest_grade)

Print ("In lowest grade:", lowest_grade)

sorted_grades = sorted(student_grades)

Print ("In sorted grades:", sorted_grades)

reversed_grades = reversed(student_grades)

Print ("In reversed grades:", reversed_grades)

grade_indices = list(range(1, num_students + 1))

Print ("In grade indices from 1 to number of students:", grade_indices)

analyze_student_grade()

6.2

Aim: To create a program to perform basic arithmetic operations based on user input

Algorithm:

1. start the program
2. Get two numbers from the user
3. Get the operation choice from the user
4. Perform the chosen operation
5. Display the result
6. stop

Program

```
def add(a,b):
```

```
    """Return the sum the two numbers."""
```

```
    return a+b
```

```
def subtract(a,b):
```

```
    """Return the difference of two numbers."""
```

```
    return a-b
```

```
def Multiply(a,b):
```

```
    """Return the product of two numbers"""
```

```
    return a*b
```

```
def divide(a,b):
```

```
    """Return the quotient of two numbers. Handles  
    division by zero"""
```

```
    return a/b
```

```
else:
```

```
    return "Error: Division by zero"
```

```
def greet(name):
```

```
    """Return a greeting message for the user."""
```

```
    return f'Hello {name}! welcome to the program.'
```

```
def main():
```

```
    num = 10
```

```
    num = 5
```

```
    print("Arithmetic operations:")
```

```
    print(f"sum of {num1} and {num2}:", add  
          (num1, num2))
```

```
    print(f"difference of {num1} and {num2}:" subtract  
          (num1, num2))
```

```
Print (f"Product of {num1} and {num2}:";multiply  
(num1,num2))
```

```
Print (f"Divide quotient of {num1} and {num2}:";divide  
(num1,num2))
```

```
User-name = "Alice".
```

```
Print ("In Greeting: ")
```

```
Print (greet(user-name))
```

```
If-name == "-main-":
```

```
main()
```

VEL TECH	
EX NO.	1
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	
TOTAL (20)	
SIGN WITH DATE	15

Result: Thus the program using 'functions' concepts was successfully executed and the output was verified

output:

Arithmetic operations

sum of 10 and 5: 15

Difference of 10 and 5: 5

Product of 10 and 5: 50

Quotient of 10 and 5: 2.0

Greeting:

Hello, Alice! Welcome to the program

10/9/25 Task-7 Implement python generator and decorators

Aim: write a python program to implement python generator and decorators

Algorithm:

Define generator function number sequence (start, end, step)

initize current value: set current to value of start

Generate sequence: while current is less than or equal to end:

- yield the current value of current
- increment current by step

Get user input: Read the starting number from user

Read the ending number from user

Read the step value from user

create generator object: create a generator object by calling number sequence (start, end, step) with user provided values

Print generated sequence: iterate over the values produced by generator object.

- Print each value

Program:

```
def number sequence (start, end, step=1)
```

```
    current = start
```

```
    while current <= end:
```

```
        yield current
```

```
        current += step
```

```
start = int (input ("Enter the starting number:"))
```

```
end = int (input ("Enter the ending number:"))
```

```
step = int (input ("Enter the step value:"))
```

```
sequence = generator = number:sequence (start, end, step)
```

output:

Enter the starting number: 1

Enter the ending number: 50

Enter the step value: 5

1

6

11

16

21

26

31

36

41

46

output:

0

1

2

for number in sequence-generator:
Print (number)

7.1 (b) Aim: Create a generator function to produce a sequence of numbers from 0 to $n-1$

Algorithm:

start function:

Define the function my-generator (n) that takes a parameter n
initialize counter ; set value to 0

Generate values : while value is less than n:

- yield the current value
- increment value by 1

Create generator object:

call my-generator (11) to create a generator object

iterate and print values

for each value produced by the generator object
print value

Program:

```
def my-generator
```

```
    value = 0
```

```
    while value < n:
```

```
        yield value
```

```
        value += 1
```

```
for value in my-generator(3)
```

```
    print(value).
```

1.2 Aim: create implement decorators for dynamic text formatting in a messaging application allowing users to choose uppercase or lowercase messages

Algorithm:

- create uppercase-decorator and lowercase-decorator
- create lowercase to convert text to lowercase
- Define shout function with @ uppercase-decorator
- Define whisper function with @ lowercase-decorator
- Define greet function to call decorated functions
- Execute greet (shout) and greet (whisper)

Program:

```
def uppercase-decorator(func):
```

```
    def wrapper(text):
```

```
        return func(text).upper()
```

```
    return wrapper
```

```
def lowercase-decorator(func):
```

```
    def wrapper(text):
```

```
        return func(text).lower()
```

```
    return wrapper
```

```
@ uppercase wrapper-decorator
```

```
def shout(text):
```

```
    return text
```

```
@ lowercase-decorator
```

```
def whisper(text):
```

```
    return text
```

output:

Hi, I AM CREATED BY A FUNCTION PASSED AS AN
ARGUMENT.

hi, i am created by a function passed as an Argument.

```
def greet(func):
```

```
    greeting = func("Hi i am created by a function  
    passed as an argument.")
```

```
    print(greeting)
```

```
greet(shout)
```

```
greet(whisper)
```

VEL TECH	
EX NO.	7
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	
TOTAL (20)	
DATE WITH DATE	15

Result: Thus a program to implement Python generator and decorators was successfully executed and output was verified.

10/09/25 Task- 8. Implement various txt/csv file operation

Aim: To write a Python program for creating and updating student registration details using txt file operations.

Algorithm:

- step 1: start
- step 2: using open() method, create and write text file "myfile.txt" with student details
- step 3: update the new registered student details using append operation in it
- step 4: open the file in read mode and using read() method print the student details
- step 5: using seek method print the particular student record
- step 6: using tell method print the current position of the file
- step 7: close the file
- step 8: stop

Program:

```
file = open("student1.txt", "w")
input1 = input("Enter columns name\n")
file.write(input1)
file.write("\n")
n = int(input("Enter the no of students"))
for i in range(0, n):
    input2 = input("Enter student's details with for new")
    file.write(input2)
    file.write("\n")
```

output:

student details using Read function is

UTU No	NAME	AGE
--------	------	-----

2305	RAM	20
------	-----	----

1920	SHIVA	21
------	-------	----

2305	RAM	20
------	-----	----

1920	SHIVA	21
------	-------	----

The length of first line is

15

output of Readline function is

2305 RAM 20

find the current position of file pointer.

29

```
file = open("student1.txt", "a")
input3 = input("Enter updated students details\n")
file.write(input3)

file = open("student1.txt", "r")
print("student details using Read function is:")
print(file.read())
print("\n")

file.seek(0)
print("The length of first line is:")
line = file.readline()
len = len(line)
print(len)
file.seek(len+1)
print("output of Reading (first student record) function is:")
print(file.readline())
print("In find the current position of file pointer:")
f = file.tell()
print(f)
file.close()
```


Result: Thus the Python program for creating and updating student registration details using txt file operations was executed successfully

8.2 Counting cases

Aim: To construct a python program whose file name is 'merge.txt' to illustrate the below content inside the file

Program: with open ("merge.txt", "w") as f:

f.write ("Python is High level language, developed
by Guido van Rossum in 1991")

with open ("merge.txt", "r") as f:

text = f.read()

upper-count = 0

lower-count = 0

digit-count = 0

for char in text:

if char.isupper():

upper-count += 1

elif char.islower():

lower-count += 1

elif char.isdigit():

print ("Uppercase letters:", upper-count)

print ("Lowercase letters:", lower-count)

print ("Digits:", digit-count)

print(f"{upper-count}, {lower-count}, {digit-count}")

input:

Filename: merge.txt

Output: 5,48,4

output:

Gaurav $\rightarrow 169.0$

Abhinav $\rightarrow 138.0$

Harvad $\rightarrow 152.0$

Ravi $\rightarrow 188.0$

8.3

Aim: Read student grades from grades.txt, calculate average grades, and write results to results.txt

Program: with open ("grades.txt", "r") as f:

```
lines = f.readlines()
n = int(lines[0].strip())
weights = lines[1].strip().split()
weights = [float(w) for w in weights]
students = []
for i in range(2, 2+n):
    parts = lines[i].strip().split()
    name = parts[0]
    marks = [int(m) for m in parts[1:]]
    total = 0
    for j in range(4):
        total = total + marks[j] * weights[j]
    students.append((name, round(total, 2)))
with open ("results.txt", "w") as f:
    for name, avg in students:
        f.write (name + " -> " + str(avg) + "\n")
print ("Average grades have been written to results.txt")
```



VEL TECH	
EX NO.	2
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	
TOTAL (20)	
DATE WITH DATE	15

Result: Thus, the Python program implement various text file operations was successfully executed and output was verified.

24/9/25 Task-9:- Implement Exceptions and Exceptional handling in Python

Aim: To implement Exceptions and Exceptional handling in Python

Algorithm

- start the Program
- initializes a list of grades (e.g; [85, 90, 78, 92, 88])
- Prompts the user to enter the index of grade
- Attempt to display the grade at specified values
- if the index is out of ranges catches the IndexError

Program

```
Grades = [85, 90, 78, 92, 88]
```

```
Print ("Grades List: %s")
```

```
try:
```

```
    index = int (input ("Enter the index of the grade they want to view:"))
```

```
    Print (f "The grade at index {index} is: {grades [index]}")
```

```
except ValueError:
```

```
    Print ("Invalid input. Please enter a numerical index.")
```

output:

Grades list: [85, 90, 78, 92, 88]

Enter a index of the grade you want to
view: 10

invalid index : Please enter a valid index

9.2 Aim: a python calculator program that performs basic arithmetic operations.

Algorithm:

1. start the program
2. Prompts the user to enter two numbers a numerator and a denominator
3. Attempts to divide the numerator by denominator
4. if the denominator is zero, catches the zero Division Error and displays an error message: "Error: Division by zero is not Allowed."

Program:

```
def divide_numbers():  
    try:  
        numerator = float(input("Enter the numerator:"))  
        denominator = float(input("Enter the denominator:"))  
        result = numerator/denominator  
        Print.(f"Result: {result}")  
    except zero Division Error:  
        print("Error: Division by zero is not allowed.")  
    except value Error:  
        Print.("Error: please enter valid numbers.")  
    divide_numbers()
```

Output:

Enter the numerator: 10

Enter the denominator: 0

ERROR!

Error: Division by zero is not allowed

output:

Enter a number: 15

Exception occurred: invalid age

Enter a number: 25

~~Excep~~ Eligible to vote

9.3 Aim: To determine a person is eligible to vote based on their age

Algorithm:-

1. Define the custom exception
2. Prompt the user for input
3. Check if the age is below 18
4. Raise an exception if the condition is met
5. Handle the exception with a custom error message

Program:-

```
class InvalidAgeException(Exception):
```

```
    "Raised when the input value is less than 18"
```

```
pass
```

```
number = 18
```

```
try:
```

```
    input - num = int(input("Enter a number:"))
```

```
    if input - num < number:
```

```
        raise InvalidAgeException
```

```
else
```

```
    print("Eligible to vote")
```

```
except InvalidAgeException:
```

```
    print("Exception occurred: Invalid age")
```

VEL TECH - CSE	
EX NO.	5
PERFORMANCE (5)	5
RESULT AND ANALYSIS (3)	3
VIVA VOCE (3)	3
RECORD (4)	4
TOTAL (15)	15
DATE	15

Result: Thus the program for implement exceptions and Exceptional handling is executed and verified successfully

8/10/25 Task-10 - use Matplotlib module for plotting in python.

Aim:- To use Matplotlib module for plotting in python

Algorithm:-

- list of programming languages
- list of corresponding popularity values
- Find max popularity .corres in list
- calculate scaling factor = $50 / \text{max-popularity}$
- calculate bar height = $\text{Popularity} * \text{scaling factor}$
- Print language + " | " + " * " * bar-height + " | " +
Popularity

Program:-

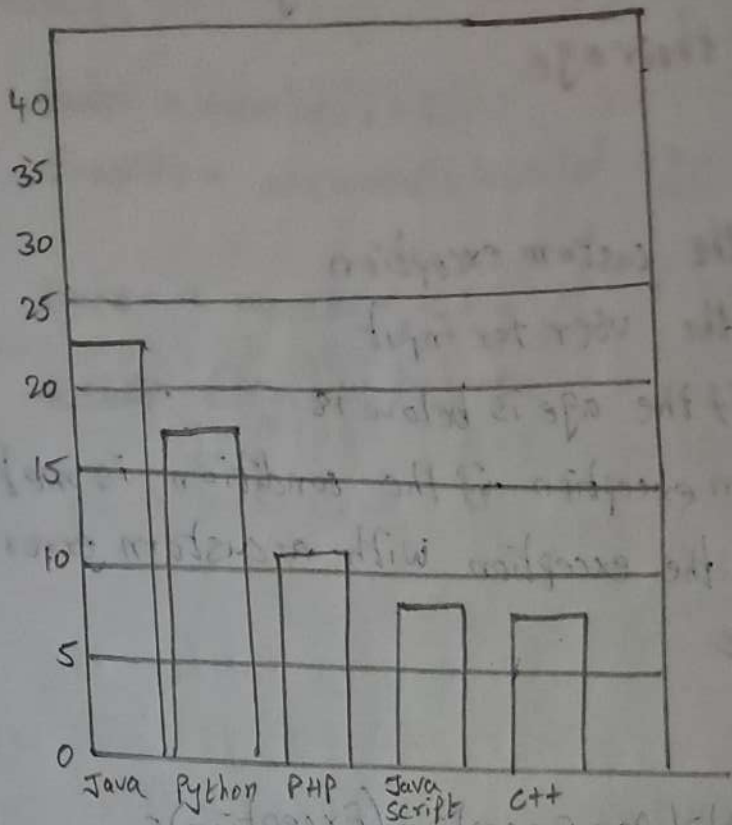
```
# pip install matplotlib
import matplotlib.pyplot as plt

languages = ["Java", "Python", "PHP", "JavaScript", "C#", "C++"]
Popularity = [22.2, 17.6, 8.8, 8, 7.7, 6.7]

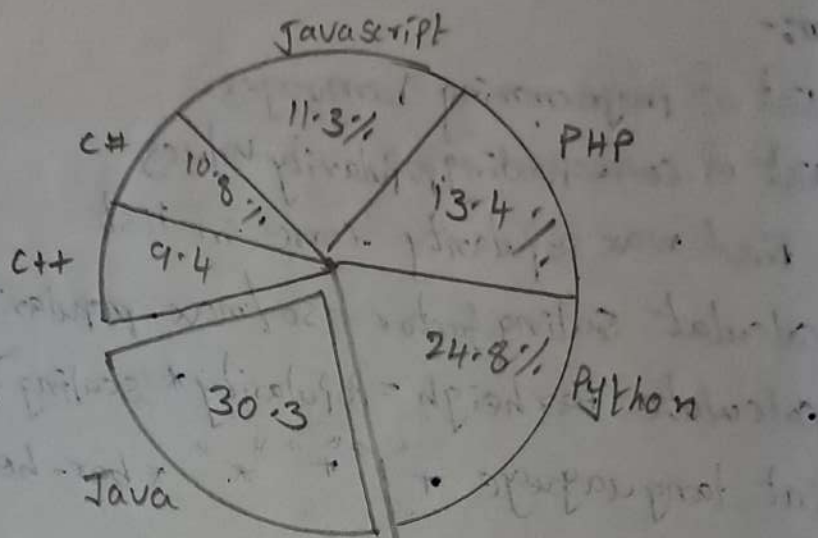
plt.bar(languages, Popularity, colour='b')
plt.title('Popularity of programming language')
plt.xlabel('Popularity')
plt.show()
```



Output:-



output:-



Aim:- To create a pie chart of popularity of programming languages

Algorithm:-

create a list of Programming languages and Popularity
create a Piechart using a matplotlib library
set a title and legend for Pie chart
show the piechart

Program:-

```
import matplotlib.pyplot as plt
```

#step 1

```
languages = ['Java', 'Python', 'PHP', 'Javascript', 'C#', 'C++']  
Popularity = [22.2, 17.6, 8.8, 7.7, 8, 6.7]
```

#step 2

```
plt.pie(Popularity, labels = languages, autopct = '%1.1f%%')
```

#step 3

```
plt.title('Popularity of Programming languages')  
plt.legend(languages, loc = "best")
```

#step 4

```
plt.show()
```

VEL TECH	
EX NO.	10
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	5
(30)	15

Result:- Thus the Program use Matplotlib module for plotting is executed and verified successfully.

15/10/22 Task 11: use Tkinter module for UI design.

Aim:- To use Tkinter module for UI design

Algorithm:-

- import tkinter module
- create a main window
- create a label with desired text
- Add label to main window using pack()
- Define a function to change font style
- create a button to call the function when clicked
- Add the button to main window using pack()
- start the main loop

Program:-

```
import tkinter as tk

# Function to change font style
def change_font():
    label.config(font=("Arial", 18, "bold"))

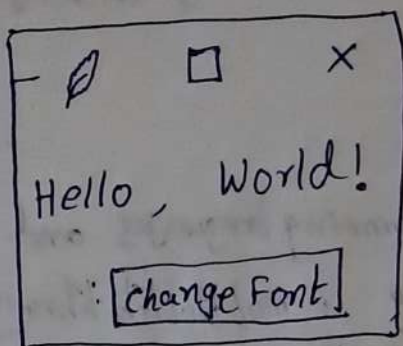
# create main window
root = tk.Tk()

# create label with desired text
label = tk.Label(root, text="Hello world!", font=
    ("Helvetica", 14))

# Add label to main window
label.pack()

# create button to change font style
button = tk.Button(root, text="change font",
    command=change_font)
```

Output :-



Add button to main window
button.pack()

start the main loop
root.mainloop()

142 Aim:- To create three single line text box to accept a value from the user tkinter module

Algorithm:- import the tkinter module

- create the main window
- Add labels and text boxes to main window
- set the size of text boxes
- create a button to submit the values
- close the main window when the button is clicked

Program:-

```
import tkinter as tk

# create the main window
root = tk.Tk()
root.title("Text-Box-input")

# create labels and text-boxes
label1 = tk.Label(root, text="Enter value 1:")
entry1 = tk.Entry(root)

label2 = tk.Label(root, text="Enter value 2:")
entry2 = tk.Entry(root)

label3 = tk.Label(root, text="Enter value 3:")
entry3 = tk.Entry(root)
```



Name

user ID

password

#set the size of text boxes

entry1.config(width=30)

entry2.config(width=30)

entry3.config(width=30)

#create a function to get values entered

def get-values():

val1 = entry1.get()

val2 = entry2.get()

val3 = entry3.get()

Print("value1:" val1)

Print("value2:" val2)

Print("value3:" val3)

#create a button to submit the values entered the textboxes submit button = tk.Button

(text="submit", command=get-values)

Add the labels text boxes and button to the main window

label1.pack()

label2.pack()

entry1.pack()

entry2.pack()

label3.pack()

entry3.pack()

submit button.pack()

Run the main event

root.mainloop()

VEL TECH	
EX NO.	
PERFORMANCE (5)	11
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	5
TOTAL (20)	
SIGN WITH DATE	15

Result: Thus The Program using Tkinter module for UI design was executed and verified successfully

15/10/25 Task-12 - simulate gaming concepts using Pygame.

Aim:- To simulate Gaming concepts using Pygame

Algorithm:-

- set the window size
- create snake class which initializes snake position
- create a fruit class which initializes fruit position
- create a function to check if snake collides with fruit and increse the score
- create a function to check & update snake position
- End the game if snake collides window

Program:-

```
# importing libraries  
import pygame  
import time  
import random
```

```
snake-speed = 15
```

```
# window size
```

```
window-x = 720
```

```
window-y = 480
```

```
# defining colours
```

```
black = pygame.color(0,0,0)
```

```
white = pygame.colour(255,255,255)
```

```
blue = pygame.color(0,0,255)
```

initializing Pygame

Pygame.init()

initialize game window

pygame.display.set-captain (heeks for heek snake)

setting default snake direction towards

right

direction = RIGHT

change to = direction

initial score

score = 0

displaying score function

def show = score (choice, color, font, size):

game over function

def game-over():

creating a text surface on which text

will be drawn

game-over-surface = my-font.render (

Your score, ' + str(score), True, red)

setting position of the text

game-over-rect.midtop = (window-x/2; window-y/4)

quit the Program

quit()

Main function

while True:

handling key events

for event in pygame.event.get():

if event.type == pygame.KEYDOWN

if event.key == pygame.K_UP

change-to = 'up'

if event.key == pygame.K_DOWN

change-to = 'down'

if event.key == pygame.K_LEFT

change-to = 'left'

if event.key == pygame.K_RIGHT

change-to = 'right'

Moving the snake

if direction == 'up'

snake_position[1] = 10

if direction == 'down'

snake_position[1] = 10

if direction == 'left'

snake_position[0] = 10

if direction == 'right'

snake_position[0] = 10

Game over conditions

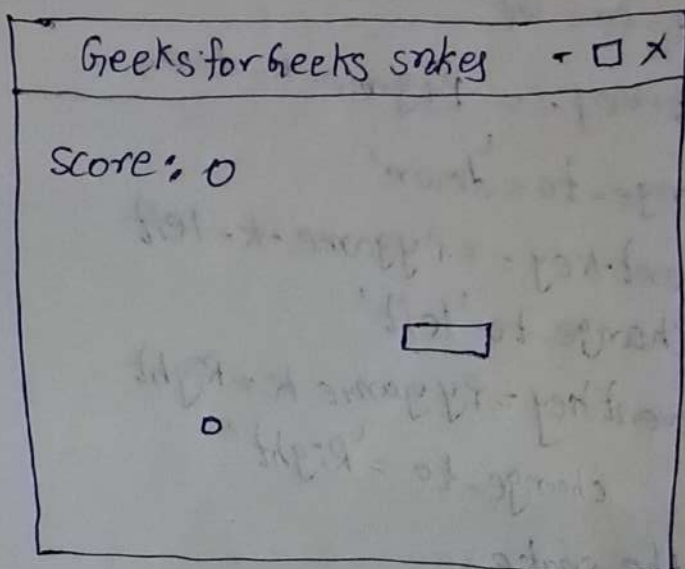
if snake_position[0] < 0 or snake_position[0] > window_x - 10:

if snake_position[0] == block[0] and

snake_position[1] == block[1]:

game_over()

output



displaying score continuously

show_score (fp, white, 'times new roman', 20)

refresh game screen

pygame.display.update()

Frame per second / refresh rate

fps, tick (snake-speed)

Completed

VEL TECH	
EX NO.	12
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	
TOTAL (20)	15
DATE	

Result:- Thus the Program for Pygame is executed and verified successfully.